RNN Sentiment Analysis

In [1]:
```
import pandas as pd
import numpy as np
import torch
```

In [2]:
```
df=pd.read_csv("IMDB Dataset.csv")
df.shape
```

Out[2]:
```
(50000, 2)
```

In [3]:
```
df.isnull().sum()
df.drop_duplicates(inplace=True)
df.shape
```

Out[3]:
```
(49582, 2)
```

Text Processing

1.Converting to lowercase

In [4]:
```
df["review"]=df["review"].str.lower()
```

2.Remove URLS

In [5]:
```
# | Part   | Meaning                 |
# | ------ | ----------------------- |
# | `http` | matches "http"          |
# | `\S`   | any non-space character |
# | `+`    | one or more             |
```

In [6]:
```
import re
def remove_urls(text):
    text = re.sub(r"http\S+" , "", text)  # (pattern, repl, string) eg - https://ww
    return text

df["review"] = df["review"].apply(remove_urls)
```

3.Removing punctuations

In [7]:
```
def remove_punctuations(text):
    text = re.sub(r"[^A-Za-z0-9\s]" , "", text) # A-Z a-z 0-9 \s
```

```
    return text

df["review"] = df["review"].apply(remove_punctuations)
```

In [8]:
```
df
```

Out[8]:

	review	sentiment
0	one of the other reviewers has mentioned that ...	positive
1	a wonderful little production br br the filmin...	positive
2	i thought this was a wonderful way to spend ti...	positive
3	basically theres a family where a little boy j...	negative
4	petter matteis love in the time of money is a ...	positive
...	...	...
49995	i thought this movie did a down right good job...	positive
49996	bad plot bad dialogue bad acting idiotic direc...	negative
49997	i am a catholic taught in parochial elementary...	negative
49998	im going to have to disagree with the previous...	negative
49999	no one expects the star trek movies to be high...	negative

49582 rows × 2 columns

4.HTML Tags remove

In [9]:
```
def remove_html(text):
    text=re.sub(r"<.*?>","",text)
    return text

df["review"] = df["review"].apply(remove_html)
```

In [10]:
```
df
```

Out[10]:

	review	sentiment
0	one of the other reviewers has mentioned that ...	positive
1	a wonderful little production br br the filmin...	positive
2	i thought this was a wonderful way to spend ti...	positive
3	basically theres a family where a little boy j...	negative
4	petter matteis love in the time of money is a ...	positive
...	...	...
49995	i thought this movie did a down right good job...	positive
49996	bad plot bad dialogue bad acting idiotic direc...	negative
49997	i am a catholic taught in parochial elementary...	negative
49998	im going to have to disagree with the previous...	negative
49999	no one expects the star trek movies to be high...	negative

49582 rows × 2 columns

5.Removing stopwords

In [11]:
```
import nltk

nltk.download("punkt")#Sentence tokenizer(divdes sentence into a list of sentences)
nltk.download("punkt_tab")
nltk.download("stopwords")
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Vedant\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Vedant\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Vedant\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[11]: True

In [12]:
```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

In [13]:
```
def remove_stopwords(text):
    tokens=word_tokenize(text)
    stop_words=stopwords.words("english")

    for word in tokens:
        if word in stop_words:
```

```
            text=text.replace(word,"")

    return text

df["review"] = df["review"].apply(remove_stopwords)
```

In [14]:
```
df
```

Out[14]:

	review	sentiment
0	e revewers nted wtchg 1 oz epode ll ho...	positive
1	wderful ltle producti br br filming techniqu...	positive
2	thought ths wderful wy spend tme o hot s...	positive
3	bsclly res fmly lttle boy jke thks res zom...	negative
4	petter mtte love time mey vully stunng fi...	positive
...	...	...
49995	thought ths move dd rght good job t wsnt s ...	positive
49996	bd plot bd dlogue bd ctng dotc drectng nnoyng...	negative
49997	ctholc tught n prochl elentry schools nuns...	negative
49998	im gog dgree previous comnt side mlt ...	negative
49999	one expects str trek moves hgh rt fns e...	negative

49582 rows × 2 columns

6.Stemming

In [15]:
```
#running->run
#played->play
#porterStemming
```

In [16]:
```
from nltk.stem import PorterStemmer

def stemming(text):
    ps=PorterStemmer()
    stemmed_words=[]

    tokens=word_tokenize(text)
    for token in tokens:
        stemmed_token=ps.stem(token)
        stemmed_words.append(stemmed_token)

    return " ".join(stemmed_words)

df["review"] = df["review"].apply(stemming)
```

In [17]:
```
df.head()
```

Out[17]:

	review	sentiment
0	e revew nted wtchg 1 oz epod ll hook y rght ex...	positive
1	wder ltle producti br br film techniqu unssum ...	positive
2	thought th wder wy spend tme o hot summer week...	positive
3	bsclli re fmli lttle boy jke thk re zomb close...	negative
4	petter mtte love time mey vulli stunng film wt...	positive

7.Encoding

In [18]:
```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df["sentiment"]=le.fit_transform(df["sentiment"])
y=df["sentiment"]
```

In [19]:
```
y
```

Out[19]:
```
0        1
1        1
2        1
3        0
4        1
        ..
49995    1
49996    0
49997    0
49998    0
49999    0
Name: sentiment, Length: 49582, dtype: int64
```

8.Vectorization

In [20]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
tf=TfidfVectorizer(max_features=5000)
X=tf.fit_transform(df["review"])
```

In [21]:
```
X
```

Out[21]:
```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 4057140 stored elements and shape (49582, 5000)>
```

9.Dataset and Dataloader

In [22]:
```
from sklearn.model_selection import train_test_split

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42
)
```

In [23]:
```
X_train.shape
```

Out[23]:
```
(39665, 5000)
```

In [24]:
```
X_test.shape
```

Out[24]:
```
(9917, 5000)
```

In [25]:
```
from torch.utils.data import TensorDataset,DataLoader
#Sparse row matrix->numparray
X_train = X_train.toarray()
X_test = X_test.toarray()
```

In [27]:
```
train_set=TensorDataset(
    torch.from_numpy(X_train).float(),#Converts NumPy array → PyTorch tensor
    torch.from_numpy(y_train.values).float()#If y_train is a pandas Series, .values
)

test_set=TensorDataset(
    torch.from_numpy(X_test).float(),
    torch.from_numpy(y_test.values).float()
)
```

In [54]:
```
#DataLoder
train_loader=DataLoader(train_set,shuffle=True,batch_size=64)
test_loader=DataLoader(test_set,shuffle=True,batch_size=64)
```

10.Build our RNN

In [55]:
```
import torch.nn as nn
import torch.optim as optim
```

In [31]:
```
# Because of batch_first=True
# (batch_size, sequence_length, input_size)
# Example:
# 32 sentences
# each sentence = 10 words
# embedding = 100
# Shape:
# (32,10,100)
```

In [32]:
```
# ------------------------------------------------------------
# FORWARD FUNCTION EXPLANATION
#
# The forward() method defines how the input data flows
```

```
# through the neural network.
#
# Input x shape:
# (batch_size, sequence_length, input_size)
#
# Example:
# batch_size = 32
# sequence_length = 10
# input_size = 50
# x shape = (32,10,50)
#
# Steps inside forward():
#
# 1. Initialize hidden state h0 with zeros
#    Shape: (num_layers, batch_size, hidden_size)
#
# 2. Pass input x and hidden state h0 into the RNN layer
#
#    out, hn = self.rnn(x, h0)
#
#    out contains hidden states for ALL timesteps
#    Shape: (batch_size, sequence_length, hidden_size)
#
# 3. Select the last timestep output
#
#    out[:, -1, :]
#
#    :  -> all batches
#    -1 -> last timestep
#    :  -> all hidden neurons
#
#    Shape becomes: (batch_size, hidden_size)
#
# 4. Pass this through the Fully Connected (Linear) layer
#
#    self.fc(hidden_size → 1)
#
#    Output shape: (batch_size, 1)
#
# 5. Return the final prediction
#
# Final Pipeline:
#
# Input → RNN → Last timestep → Fully Connected → Output
# ------------------------------------------------------------
```

In [56]:

```
class RNN(nn.Module):
    def __init__(self,input_size,hidden_size=128,num_layers=1):
        super().__init__()

        self.hidden_size=hidden_size
        self.num_layers=num_layers

        # RNN layer
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
```

```
        #Fully connected layer
        self.fc=nn.Linear(hidden_size,1)

    def forward(self,x):
         # optional => shape (num of layers, batch size, hidden size)
        h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size)

        out, _ = self.rnn(x, h0)
        # 1st value = hidden state of all the timesteps => (batch, seq_len, hidden
        # 2nd value = final hidden state of last timestep

        out = self.fc(out[:, -1, :])
        return out
```

11.Optimizer and loss function

In [57]:
```
input_size = X_train.shape[1]

model = RNN(input_size)

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters())
```

12.Training

In [60]:
```
epochs=10

for epoch in range(epochs):
    model.train()

    for Xb,yb in train_loader:
        optimizer.zero_grad()

        # adding a new dimension to input
        # required because RNN expects 3D input:
        # (batch_size, sequence_length, input_size)
        Xb=Xb.unsqueeze(1)

        outputs=model(Xb)#(batch_size,1)

        outputs=torch.sigmoid(outputs.squeeze())#(batch_size,)->probability

        loss=criterion(outputs,yb)#Compute loss
        loss.backward()#backpropgation
        optimizer.step()#weight updates
    print(f"epoch = {epoch+1}/{epochs} and loss = {loss.item()}")
```

```
epoch = 1/10 and loss = 0.23398496210575104
epoch = 2/10 and loss = 0.1339002102613449
epoch = 3/10 and loss = 0.15321119129657745
epoch = 4/10 and loss = 0.1395319253206253
epoch = 5/10 and loss = 0.106726735830307
epoch = 6/10 and loss = 0.2075696438550949
epoch = 7/10 and loss = 0.27737995982170105
epoch = 8/10 and loss = 0.3331489562988281
epoch = 9/10 and loss = 0.20726701617240906
epoch = 10/10 and loss = 0.2654903531074524
```

13.Evalution

In [61]:

```
model.eval()

with torch.no_grad():
    correct_vals=0
    tot_vals=0

    for Xb,yb in test_loader:
        Xb=Xb.unsqueeze(1)

        outputs=model(Xb)
        predicted=(torch.sigmoid(outputs.squeeze())>0.5).float()

        tot_vals+=yb.size(0)
        correct_vals+=(predicted==yb).sum().item()

    print(f"accuracy = {correct_vals/tot_vals*100}")
```

```
accuracy = 85.46939598668952
```